Institut für Parallele und Verteilte Systeme

Universität Stuttgart
Universitätsstraße 38
D–70569 Stuttgart

Masterarbeit

Connecting MetaConfigurator to the Semantic Web: RDF Authoring, Ontology Integration, and Knowledge Graph Support.

Mahdi Jafarkhani

Studiengang:	Computer Science
Prüfer/in:	Jun. Prof. Dr. rer.nat. Benjamin Uekermann
Betreuer/in:	Felix Neubauer, M.Sc.
Beginn am:	October 15, 2025
Beendet am:	April 15, 2026

Abstract

Scientific workflows increasingly produce structured JavaScript Object Notation (JSON) data that is easy to exchange but difficult to interpret semantically across systems. While JSON Schema supports structural validation, it does not provide native semantics for Linked Data integration. This thesis addresses that gap by extending MetaConfigurator, a schema-driven web application for structured data authoring, with Semantic Web capabilities.

The core contribution is an integrated Resource Description Framework (RDF) Authoring Panel that supports end-to-end semantic data workflows in a single User Interface (UI). The extension enables transformation of JSON documents into JavaScript Object Notation for Linked Data (JSON-LD) using RDF Mapping Language (RML), direct authoring and editing of semantic structures, SPARQL Protocol and RDF Query Language (SPARQL)-based querying, and interactive knowledge graph visualization. In addition, Artificial Intelligence (AI)-assisted generation of SPARQL queries and RML mappings from user hints is introduced to reduce the entry barrier for non-expert users.

The result is a unified environment that bridges schema-driven configuration editing and Semantic Web technologies. By combining validation, transformation, querying, and visualization in one workflow, the approach improves interoperability of scientific experiment data and supports more accessible creation and exploration of semantically enriched knowledge graphs.

Kurzfassung

Wissenschaftliche Workflows erzeugen zunehmend strukturierte JSON-Daten, die sich zwar leicht austauschen lassen, deren semantische Interpretation über Systemgrenzen hinweg jedoch schwierig ist. JSON Schema ermöglicht strukturelle Validierung, bietet aber keine nativen Semantiken für die Integration in Linked-Data-Infrastrukturen. Diese Arbeit schließt diese Lücke, indem MetaConfigurator, eine schema-getriebene Webanwendung zur Bearbeitung strukturierter Daten, um Semantic-Web-Funktionen erweitert wird.

Der zentrale Beitrag ist ein integriertes RDF-Authoring-Panel, das durchgängige semantische Daten-Workflows in einer einzigen Benutzeroberfläche unterstützt. Die Erweiterung umfasst die Transformation von JSON-Dokumenten nach JSON-LD mittels RML, die direkte Erstellung und Bearbeitung semantischer Strukturen, SPARQL-basierte Abfragen sowie eine interaktive Visualisierung von Wissensgraphen. Ergänzend wird eine KI-gestützte Generierung von SPARQL-Abfragen und RML-Mappings aus Nutzerhinweisen eingeführt, um die Einstiegshürde für Nicht-Expert:innen zu senken.

Als Ergebnis entsteht eine einheitliche Umgebung, die schema-getriebene Konfigurationsbearbeitung mit Semantic-Web-Technologien verbindet. Durch die Kombination von Validierung, Transformation, Abfrage und Visualisierung in einem Workflow verbessert der Ansatz die Interoperabilität wissenschaftlicher Versuchsdaten und erleichtert die Erstellung sowie Exploration semantisch angereicherter Wissensgraphen.

Contents

List of Figures

List of Tables

Abbreviations

Notation	Description	Page List
AI	Artificial Intelligence	3, 28, 30, 31, 33, 36, 37, 46, 47, 59
CEDAR	Center for Expanded Data Annotation and Retrieval	31
CSV	comma-separated values	22, 36
ETL	extract, transform, load	31
GUI	graphical user interface	24, 26, 41
IRI	Internationalized Resource Identifier	21, 22, 29, 30, 46
JSON	JavaScript Object Notation	3, 4, 15, 16, 19, 21, 22, 24, 26–31, 33, 35–37, 44, 46, 53, 57, 59

Notation	Description	Page List
JSON-LD	JavaScript Object Notation for Linked Data	3, 4, 15, 19–22, 24, 28–31, 33, 35–37, 41, 42, 44–47, 53, 59
JSONPath	JSON Path	24
LLM	Large Language Model	46
MOF	Metal-Organic Framework	53
OWL	Web Ontology Language	31, 33
QUDT	Quantities, Units, Dimensions, and Data Types	19–21
R2RML	RDB to RDF Mapping Language	22, 33, 36
RDF	Resource Description Framework	3, 4, 18–22, 24, 28–31, 33, 35–37, 41, 42, 44–46, 53, 57, 59
RDFLib	RDFLib	31
RML	RDF Mapping Language	3, 4, 15, 22, 24, 29, 30, 33, 36, 37, 46, 47, 53, 57, 59

Notation	Description	Page List
SHACL	Shapes Constraint Language	19, 20, 31
SPARQL	SPARQL Protocol and RDF Query Language	3, 4, 19, 21, 29–31, 33, 41, 46, 47, 59
SQL	Structured Query Language	24
UI	User Interface	3, 24
URI	Uniform Resource Identifier	27
W3C	World Wide Web Consortium	19, 22
XML	Extensible Markup Language	22, 36
XPath	XML Path Language	24
YAML	YAML Ain't Markup Language	24

1 Background and Related Work

The increasing adoption of computational experiments and simulation workflows in scientific research has resulted in large volumes of structured data describing parameters, metrics, and results. Ensuring that such data can be validated, exchanged, and semantically interpreted requires standardized mechanisms for describing structure and meaning. This chapter introduces three key technologies that enable this process: JSON for structured data representation, JSON Schema for structural validation, and semantic mappings using RML to produce semantically enriched JSON-LD representations. The discussion is illustrated using a running example from a simulation experiment where the element size parameter influences a computed stress metric.

1.1 JSON and JSON Schema

1.1.1 JSON as a Data Representation Format

JSON is a lightweight data-interchange format widely used for representing structured data in web services, scientific workflows, and configuration systems. JSON encodes data as attribute-value pairs and ordered lists, making it human-readable and easy to parse by machines. Due to its simplicity and broad ecosystem support, JSON has become a de facto standard for structured data exchange across many domains [Bra17].

In computational science workflows, JSON files often capture experiment parameters and resulting metrics. The following example illustrates a minimal JSON document describing the input parameter `element_size` and the resulting metric `max_von_mises_stress`.

In Listing 1.1, each entry in the `parameters` array contains a `name` (e.g., `element_size`), a numeric `value` (`0.025`), and a `unit` (`m`), while each entry in the `metrics` array contains a result variable such as `max_von_mises_stress`. [URR+26]

In our benchmark example, `element_size` represents a meshing property that controls mesh density for h-refinement, i.e., the characteristic size of finite elements. The `max_von_mises_stress` value is a scalar result metric defined as the maximum von Mises stress over the simulated domain. [URR+26]

```json
{
  "parameters": [
    {
      "name": "element_size",
      "value": 0.025,
      "unit": "M"
    }
  ],
  "metrics": [
    {
      "name": "max_von_mises_stress",
      "value": 299791507.5586333,
      "unit": "MPa"
    }
  ]
}
```

Listing 1.1: Example JSON representation of simulation input parameters and results

While JSON provides a convenient serialization format, it does not inherently enforce structural constraints. Therefore, additional mechanisms are required to validate that JSON documents conform to an expected structure.

1.1.2 JSON Schema for Data Validation

JSON Schema provides a standardized mechanism for describing the expected structure and constraints of JSON documents. A JSON Schema defines properties, data types, required attributes, and validation rules for JSON instances. This allows automated validation of data before further processing or transformation. The JSON Schema specification defines a vocabulary for describing JSON documents and their validation rules [WAHD20].

Listing 1.2 shows a JSON Schema corresponding to the example JSON document in Listing 1.1. The schema ensures that the JSON document contains the expected parameter and metric fields and enforces the correct numeric data types.

The schema in Listing 1.2 directly mirrors the example instance in Listing 1.1. It requires the two top-level arrays (`parameters` and `metrics`), where each entry is defined by a common structure containing the fields `name`, `value`, and `unit`. The schema enforces that each variable has a textual identifier (`name`), a numeric value (`value`), and an associated unit (`unit`).

Using JSON Schema enables consistent validation of experiment data and ensures that all required attributes are present before transformation into other representations.

However, JSON Schema focuses primarily on structural validation and does not provide semantic meaning or interoperability with knowledge graph technologies. To address this limitation, semantic mapping techniques can be applied.

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "title": "Simulation Results",
  "type": "object",
  "properties": {
    "parameters": {
      "type": "array",
      "items": {
        "$ref": "#/$defs/variable"
      }
    },
    "metrics": {
      "type": "array",
      "items": {
        "$ref": "#/$defs/variable"
      }
    }
  },
  "required": ["parameters", "metrics"],
  "additionalProperties": false,
  "$defs": {
    "variable": {
      "type": "object",
      "properties": {
        "name": {
          "type": "string",
          "description": "Name of the parameter or metric"
        },
        "value": {
          "type": "number",
          "description": "Numeric value of the parameter or metric"
        },
        "unit": {
          "type": "string",
          "description": "Unit symbol (e.g., m, MPa)"
        }
      },
      "required": ["name", "value", "unit"],
      "additionalProperties": false
    }
  }
}
```

Listing 1.2: JSON Schema validating the simulation result for JSON document in Listing 1.1

1.2 Semantic Web

1.2.1 Ontology: Modeling Knowledge Domains

Ontologies provide a formal specification of concepts and relationships within a domain of knowledge. One of the most widely cited and influential definitions is provided by Gruber [Gru93]:

> "An ontology is an explicit specification of a shared conceptualization."

In this definition, the key terms can be interpreted as follows:

- **Explicit**: The concepts and relations are stated clearly and unambiguously, rather than being left implicit in code or documentation.
- **Specification**: The concepts are described in a formal, structured model (e.g., classes, properties, constraints, and axioms) that can be processed consistently by both humans and machines.
- **Shared conceptualization**: The model captures a common understanding of a domain agreed upon by a community, allowing data created by different people or systems to be interpreted in the same way.

Later work further emphasizes that ontologies should be formal and shared in order to support machine interpretation and interoperability across communities [Gua98; SBF98]. They enable shared understanding among humans and machines by defining classes, properties, and axioms that describe entities and their interrelations.

In the context of scientific experiments, ontologies can define experiment parameters, measurement units, procedures, and results. For example, an ontology might define a class `PropertyValue` representing any measured property, a class `Method` representing experimental procedures, and relationships such as `hasParameter` and `investigates` to link methods to parameters and observed metrics.

Ontologies serve as the backbone for semantic data integration and reasoning. They allow data from heterogeneous sources to be interpreted consistently, facilitate interoperability, and enable advanced queries across datasets using formal reasoning engines.

1.2.2 Semantic Web Concepts

The Semantic Web extends the World Wide Web by enabling data to be represented in a machine-readable, semantically rich form. Data is described using standards such as RDF, which encodes information as triples consisting of subject, predicate, and object. This allows heterogeneous data to be interlinked and queried using semantic technologies [BHL01].

JSON-LD is one of the key technologies bridging conventional JSON data and the Semantic Web. By adding semantic annotations, JSON-LD enables JSON documents to be interpreted as RDF graphs. For instance, in the example simulation data (Listing 1.3), the simulation parameter `element_size` is represented as a `schema:PropertyValue` class.

```
{
  "@id": "local:variable_element_size",
  "@type": "schema:PropertyValue",
  "rdfs:label": "element_size",
  "schema:unitCode": {
    "@id": "unit:M"
  },
  "schema:value": {
    "@type": "http://www.w3.org/2001/XMLSchema#double",
    "@value": "2.5E-2"
  }
}
```

Listing 1.3: Example of semantic annotation of a parameter `element_size`

In this JSON-LD snippet, the parameter `element_size` is represented as a resource of type `schema:PropertyValue`. Its numeric value is stored in `schema:value` and the associated unit is linked via `schema:unitCode` to the Quantities, Units, Dimensions, and Data Types (QUDT) unit `m`, which defines the meter. This representation allows applications and knowledge graph systems to unambiguously interpret the measurement and integrate it with other semantically annotated data. Similarly, other variables such as metrics (e.g., `max_von_mises_stress`) follow the same pattern, and experiment methods are linked to ontology classes such as `m4i:Method`.

Semantic Web technologies thus provide a framework for:

- Interoperable data representation across different systems and domains.
- Linking and reasoning over experimental data using ontologies.
- Querying and integrating datasets through standard languages like SPARQL.

By grounding JSON data in ontologies and RDF through JSON-LD, this simulation result becomes part of the Semantic Web, enabling more advanced analysis, automated reasoning, and integration with other scientific knowledge graphs.

1.2.3 SHACL: Shapes Constraint Language

The Shapes Constraint Language (SHACL) is a World Wide Web Consortium (W3C) standard for validating RDF graphs against a set of structural and semantic constraints [W3C17]. SHACL allows ontology-based validation of RDF data by defining ”shapes”that describe the expected properties, data types, and cardinality of RDF nodes.

For example, consider the simulation parameter `element_size` in the JSON-LD example (Listing 1.3). Using SHACL, we can define a shape that enforces that every `schema:PropertyValue` representing an experiment parameter must have a numeric `schema:value` and a `schema:unitCode` pointing to a valid unit from the QUDT ontology:

```
@prefix sh: <http://www.w3.org/ns/shacl#> .
@prefix schema: <https://schema.org/> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
@prefix unit: <http://qudt.org/vocab/unit/> .
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .
@prefix local: <https://www.example.org/> .

local:PropertyValueShape
    a sh:NodeShape ;
    sh:targetClass schema:PropertyValue ;

    # The variable name label
    sh:property [
        sh:path rdfs:label ;
        sh:datatype xsd:string ;
        sh:minCount 1 ;
        sh:maxCount 1 ;
    ] ;

    # The numeric value of the parameter or metric
    sh:property [
        sh:path schema:value ;
        sh:datatype xsd:double ;
        sh:minCount 1 ;
        sh:maxCount 1 ;
    ] ;

    # The unit, as a named QUDT IRI
    sh:property [
        sh:path schema:unitCode ;
        sh:nodeKind sh:IRI ;
        sh:minCount 1 ;
        sh:maxCount 1 ;
    ] .
```

Listing 1.4: SHACL shape for validating simulation parameters

By applying this shape, RDF data can be automatically checked for compliance with the expected structure and semantics, preventing errors or inconsistencies before integration into a knowledge graph.

1.2.4 SPARQL: Querying Semantic Data

SPARQL is the standard query language for RDF datasets [PS13]. SPARQL allows retrieval, filtering, and aggregation of data stored in knowledge graphs. Queries are expressed in terms of triples, similar to RDF statements.

Listing 1.5 presents an SPARQL example using the JSON-LD simulation data, where all experiment parameters and metrics are retrieved together with their corresponding values and units.

```
PREFIX schema: <https://schema.org/>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
PREFIX unit: <http://qudt.org/vocab/unit/>
PREFIX local: <https://www.example.org/>

SELECT ?parameter ?value ?unit
WHERE {
    ?param a schema:PropertyValue ;
           rdfs:label ?parameter ;
           schema:value ?value ;
           schema:unitCode ?unit .
}
```

Listing 1.5: SPARQL query retrieving parameter values and units

This query returns the `element_size` parameter and `max_von_mises_stress` along with their numeric values and associated QUDT units (`unit:M` and `unit:MPa`), reflecting the structure of the JSON-LD representation where parameters and metrics are modeled as `schema:PropertyValue` resources.

This demonstrates how semantic annotations enable precise, ontology-aware data access. SPARQL supports complex operations such as joins, filters, and reasoning over ontologies, making it an essential tool for exploring and analyzing semantically enriched simulation and experiment data.

1.2.5 Semantic Data Representation using JSON-LD

JSON-LD extends JSON by introducing semantic annotations that allow data to be interpreted as linked data within RDF [SKL14]. JSON-LD integrates JSON with established Semantic Web standards by providing mechanisms for defining contexts, identifiers, and relationships between entities.

A typical JSON-LD document has two key structural components:

- **@context:** This section defines the mapping between JSON keys and ontology Internationalized Resource Identifiers (IRIs) or vocabularies. It provides semantic meaning to each field and enables JSON documents to be interpreted as RDF triples.

- **@graph:** This array contains the actual data entities, each represented as a node with an `@id`, a `@type`, and associated properties. Relationships between entities are expressed using references to other `@id` values, forming a connected graph structure.

Listing 1.6 shows a JSON-LD representation corresponding to the example simulation result. The `@context` defines prefixes such as `schema` and `unit`, while the `@graph` contains nodes representing the simulation run, the `element_size` parameter, and the stress metric.

This structure allows each entity to be unambiguously identified, linked, and queried using RDF-aware tools. By separating semantic definitions (`@context`) from the data graph (`@graph`), JSON-LD documents support both human readability and machine interoperability, enabling integration with knowledge graphs and other Semantic Web infrastructures.

1.2.6 RML for Mapping JSON to RDF

RML is a declarative language for transforming heterogeneous data sources into RDF representations. RML extends the W3C-standardized RDB to RDF Mapping Language (R2RML) language and supports mappings from JSON, Extensible Markup Language (XML), comma-separated values (CSV), and relational databases into RDF datasets [DVC+14].

Formally, an RDF triple is a 3-tuple:

$$(s, p, o) \in (U \cup B) \times U \times (U \cup B \cup L)$$

where:

- U is the set of all IRIs,
- B is the set of blank nodes (anonymous resources),
- L is the set of RDF literals (e.g., strings, numbers, dates),
- $s \in U \cup B$ is the subject,
- $p \in U$ is the predicate (also called property),
- $o \in U \cup B \cup L$ is the object.

An RDF graph G is then defined as a finite set of such triples:

$$G = \{(s_i, p_i, o_i) \mid i = 1, \ldots, n\}, \quad s_i \in U \cup B,\ p_i \in U,\ o_i \in U \cup B \cup L$$

RML mappings are themselves RDF graphs specifying how elements from a source data structure are transformed into RDF triples. Each mapping defines:

```
{
    "@context": {
        "m4i": "http://w3id.org/nfdi4ing/metadata4ing#",
        "schema": "https://schema.org/",
        "rdfs": "http://www.w3.org/2000/01/rdf-schema#",
        "unit": "http://qudt.org/vocab/unit/",
        "local": "https://www.example.org/"
    },
    "@graph": [
        {
            "@id": "local:run_fenics_simulation",
            "@type": "m4i:Method",
            "m4i:hasParameter": {
                "@id": "local:variable_element_size"
            },
            "m4i:investigates": {
                "@id": "local:variable_max_von_mises_stress"
            }
        },
        {
            "@id": "local:variable_element_size",
            "@type": "schema:PropertyValue",
            "rdfs:label": "element_size",
            "schema:unitCode": {
                "@id": "unit:M"
            },
            "schema:value": {
                "@type": "http://www.w3.org/2001/XMLSchema#double",
                "@value": "2.5E-2"
            }
        },
        {
            "@id": "local:variable_max_von_mises_stress",
            "@type": "schema:PropertyValue",
            "rdfs:label": "max_von_mises_stress",
            "schema:unitCode": {
                "@id": "unit:MPa"
            },
            "schema:value": {
                "@type": "http://www.w3.org/2001/XMLSchema#double",
                "@value": "2.997915075586333E8"
            }
        }
    ]
}
```

Listing 1.6: JSON-LD representation of the simulation result presented in Listing 1.1

- **Logical source:** the data source and reference formulation (e.g., JSON Path (JSONPath), XML Path Language (XPath), Structured Query Language (SQL) query),
- **Subject map:** rules to generate the *subject* of triples,
- **Predicate-object maps:** rules to generate the *predicate* and *object* for triples.

Listing 1.7 shows part of an example RML mapping that converts the JSON simulation output from Listing 1.1 into the JSON-LD structure presented in Listing 1.6.

As one concrete example, `<#ParameterMapping>` iterates over the array `$.parameters[*]` in the JSON document. For each entry, it creates an RDF resource with an IRI constructed from the `name` field (e.g., `local:variable_element_size`) and assigns it the type `schema:PropertyValue`. The JSON attribute `name` is mapped to `rdfs:label`, while the numeric field `value` is mapped to `schema:value`. The unit is converted into a QUDT unit IRI by constructing the resource `http://qudt.org/vocab/unit/{unit}` and assigning it via `schema:unitCode`.

Through this mapping process, structured simulation outputs are automatically converted into RDF triples forming a semantically enriched knowledge graph.

1.3 MetaConfigurator for Schema-Driven Data Modeling

The technologies discussed in the previous sections enable structured, validated, and semantically enriched data representations. However, creating and editing such structured data manually can be challenging, particularly for users who are not familiar with textual formats such as JSON or with schema-based validation mechanisms. To address this challenge, tools that support schema-driven data modeling and editing are required.

MetaConfigurator[1] is an open-source web application designed to simplify the creation, editing, and management of structured data and schemas [NBX+24; NPU25]. The tool generates graphical user interfaces (GUIs) dynamically based on a provided schema, allowing users to interact with structured data through intuitive forms rather than manually editing raw text files. By leveraging JSON Schema as the underlying model description, MetaConfigurator enables consistent data validation and guided data entry.

MetaConfigurator follows a schema-driven approach: the structure of the UI is derived directly from the schema that describes the data model. This design allows the same tool to be applied across different domains without the need to implement domain-specific UIs. The application runs entirely in the user's browser and supports multiple structured data formats such as JSON and YAML Ain't Markup Language (YAML) [NBX+24; NPU25].

[1] `https://www.metaconfigurator.org`

```
@prefix rr: <http://www.w3.org/ns/r2rml#> .
@prefix rml: <http://semweb.mmlab.be/ns/rml#> .
@prefix ql: <http://semweb.mmlab.be/ns/ql#> .
@prefix m4i: <http://w3id.org/nfdi4ing/metadata4ing#> .
@prefix schema: <https://schema.org/> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
@prefix unit: <http://qudt.org/vocab/unit/> .
@prefix local: <https://www.example.org/> .

<#RunFenicsSimulation>
  rml:logicalSource [
    rml:source "Data.json" ;
    rml:referenceFormulation ql:JSONPath ;
    rml:iterator "$"
  ] ;

  rr:subjectMap [
    rr:template "https://www.example.org/run_fenics_simulation" ;
    rr:class m4i:Method
  ] ;

  rr:predicateObjectMap [
    rr:predicate m4i:hasParameter ;
    rr:objectMap [ rr:parentTriplesMap <#ParameterMapping> ]
  ] ;

  rr:predicateObjectMap [
    rr:predicate m4i:investigates ;
    rr:objectMap [ rr:parentTriplesMap <#MetricMapping> ]
  ] .

<#ParameterMapping>
  rml:logicalSource [
    rml:source "Data.json" ;
    rml:referenceFormulation ql:JSONPath ;
    rml:iterator "$.parameters[*]"
  ] ;

  rr:subjectMap [
    rr:template "https://www.example.org/variable_{name}" ;
    rr:class schema:PropertyValue
  ] ;

  rr:predicateObjectMap [
    rr:predicate rdfs:label ;
    rr:objectMap [ rml:reference "name" ]
  ] ;

  rr:predicateObjectMap [
    rr:predicate schema:value ;
    rr:objectMap [ rml:reference "value" ]
  ] ;

  rr:predicateObjectMap [
    rr:predicate schema:unitCode ;
    rr:objectMap [
      rr:template "http://qudt.org/vocab/unit/{unit}" ;
      rr:termType rr:IRI
    ]
  ] .
...
```

Listing 1.7: Subset of a RML mapping which converts JSON data in Listing 1.1 to JSON-LD in Listing 1.6

This tool is organized into three main views: Data Editor, Schema Editor, and Settings [NPU25]. In each view, users can work in a synchronized split interface combining a text editor and a generated GUI, which supports both expert-oriented direct editing and guided editing for non-experts.

1.3.1 Core Features

MetaConfigurator provides several features that facilitate the creation and maintenance of structured data models:

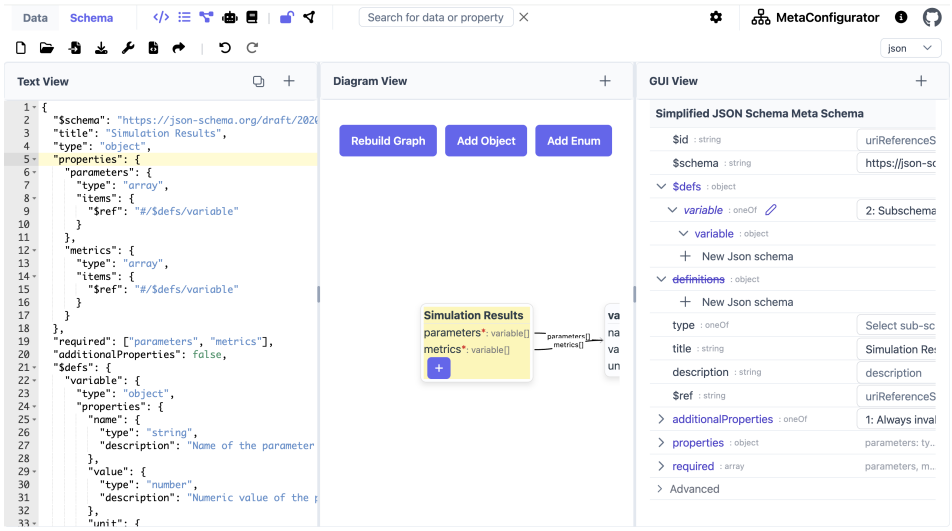

Figure 1.1: A snapshot of MetaConfigurator's schema editing interface, showing the JSON Schema from Listing 1.2

- **Visual Schema Editor:** Users can create and modify JSON schemas using a graphical interface, enabling intuitive construction of data models without manually writing schema definitions. Figure 1.1 depicts an example of schema editing in MetaConfigurator, showing the JSON Schema from Listing 1.2. The left panel shows the raw JSON Schema, the middle panel provides a diagram view for editing, and the right panel offers a structured form for schema modification.
- **Simplified and Advanced Schema Modes:** To reduce complexity while preserving expressiveness, MetaConfigurator provides configurable schema-editing modes. A simplified mode hides advanced JSON Schema options, while an advanced mode exposes the full feature set. This behavior is implemented through a meta-schema-builder approach that dynamically derives the editor schema from user settings [NPU25].

- **Schema-Driven Form Generation:** Based on a given JSON Schema, MetaConfigurator automatically generates a graphical form for editing instance data. This ensures that all entered data conforms to the defined structure and constraints.
- **Integrated Text and GUI Editing:** The tool combines a traditional text editor with a graphical editor, allowing users to switch between raw JSON editing and structured form-based editing within the same interface. Figure 1.2 depicts an example of data editing in MetaConfigurator, showing the JSON instance from Listing 1.1 that conforms to the JSON Schema in Listing 1.2. The left panel displays the raw JSON data, while the right panel offers a structured form for editing.
- **Schema Validation Support:** During data editing, MetaConfigurator validates instance data against the provided JSON Schema, ensuring structural correctness and preventing invalid data entries.
- **Schema Visualization and Navigation:** Complex schemas can be explored using a tree-like or diagram-based representation, helping users understand hierarchical structures and relationships between properties. The 2025 extension introduces an interactive diagram-like schema view with synchronized editing actions (e.g., renaming and adding properties), while advanced constructs such as compositions and conditionals are primarily visualized rather than fully edited graphically [NPU25].
- **CSV Import and Schema Inference:** Tabular data can be imported from CSV/Excel-oriented workflows into JSON, with automatic initial schema inference to reduce manual modeling effort [NPU25].
- **Snapshot Sharing and URL-Based Collaboration:** MetaConfigurator supports sharing complete editor states (data, schema, settings) through shareable links and snapshot storage, enabling reproducible collaboration between users [NPU25].
- **Limited Ontology Integration:** While the tool provides a JSON-LD export that adds `@context` and `@id` fields, its support for ontologies is still quite limited. It enables Uniform Resource Identifier (URI)-based references and offers auto-completion for ontology concepts (via a specified ontology endpoint) during structured data modeling, but more advanced ontology integration is not supported.

The above extensions were demonstrated in a chemistry use case, where tabular experiment data was migrated from CSV/Excel-oriented workflows to JSON, extended with additional metadata, and iteratively refined through schema editing [NPU25]. These capabilities significantly reduce the effort required to design and maintain structured data models and improve accessibility for users who may not be familiar with schema languages.

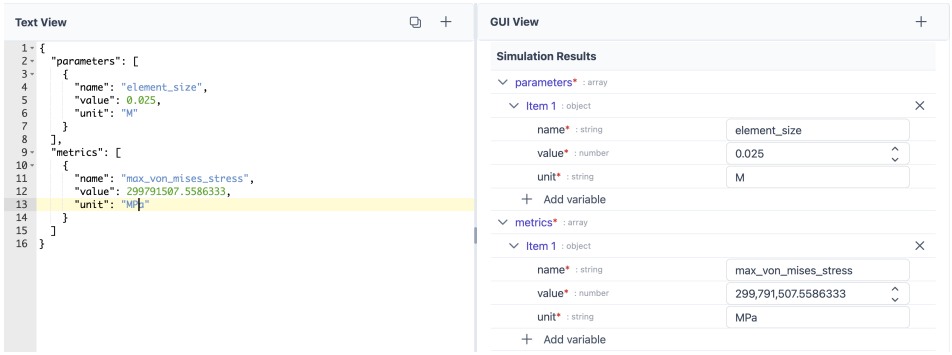

Figure 1.2: A snapshot of MetaConfigurator's data editing interface, showing the JSON instance from Listing 1.1

1.3.2 AI-Assisted Schema Engineering and Mapping

Recent work extends MetaConfigurator with AI-assisted support for schema creation, schema modification, and schema mapping based on natural-language user input [NUP25]. The approach combines large language models with deterministic safeguards to improve reliability and usability.

For schema authoring, MetaConfigurator provides a chat-like interface where users describe their intent in natural language. The system then performs prompt construction and context management automatically, including role instructions, task-specific formatting constraints, and relevant schema context. Generated results are directly validated and visualized in the schema editor, and users remain in control through a human-in-the-loop workflow where generated output can be reviewed and manually refined before acceptance.

For data transformation, the system generates mapping expressions from user input and executes these mappings deterministically. In the presented implementation, JSONata expressions are generated from an input document and a target schema, then validated and executed in a controlled pipeline. This separates AI-based generation from rule execution and supports scalable transformation workflows across heterogeneous source formats.

1.3.3 Limitations for Semantic Web Integration

Despite its strong support for schema-driven data modeling, MetaConfigurator currently provides only limited support for Semantic Web technologies: while the platform enables the creation and validation of structured data using JSON and JSON Schema, it does not natively support Linked Data representations such as JSON-LD or RDF graphs.

As discussed in previous sections, the Semantic Web relies on standards such as RDF, ontologies, and IRIs to represent data in a machine-interpretable and interoperable manner. JSON-LD serves as an important bridge between traditional JSON-based data structures and RDF-based knowledge graphs. However, MetaConfigurator currently treats JSON documents primarily as structured configuration data and does not interpret or expose their underlying RDF semantics.

Several limitations arise from this restriction:

1. **Lack of JSON-LD context authoring:** Users cannot directly author or edit JSON-LD contexts (e.g., the `@context` section that defines mappings between JSON keys and ontology IRIs). Without such support, linking data elements to ontologies requires manual editing outside the tool.
2. **No direct inspection or editing of RDF triples:** MetaConfigurator does not provide mechanisms for inspecting or manipulating RDF triples derived from the underlying JSON data. As a result, users cannot directly view the semantic graph structure implied by JSON-LD documents.
3. **Missing transformation from JSON to RDF:** The platform lacks built-in mechanisms for transforming conventional JSON documents into semantic representations. As discussed in Section 1.2.6, technologies such as RML enable declarative transformations from JSON to RDF, but such transformations are not currently integrated into MetaConfigurator's workflow.
4. **Lack of semantic querying capabilities:** MetaConfigurator does not support semantic querying of RDF data. As described in Section 1.2.4, SPARQL provides a standard mechanism for querying RDF graphs and retrieving structured information from knowledge graphs.
5. **No visualization of RDF knowledge graphs:** Knowledge graphs are often easier to understand when represented visually as nodes and relationships. However, the current platform does not provide mechanisms to visually explore RDF triples or inspect the structure of the resulting knowledge graph.

To address these limitations, this thesis extends MetaConfigurator with an **RDF Authoring Panel** that integrates Semantic Web technologies directly into the schema-driven editing environment. The main contributions of this thesis include:

1. **Transformation of JSON documents to JSON-LD:** The system enables the transformation of conventional JSON documents into JSON-LD using RML mappings. This allows structured JSON data created within MetaConfigurator to be converted into RDF triples that can be integrated into Semantic Web infrastructures.

2. **RDF authoring and editing support:** The RDF Authoring Panel allows users to directly inspect and edit RDF data. This includes editing the JSON-LD `@context` as well as creating and modifying RDF triples within the interface.
3. **Ontology-aware IRI auto-completion:** The system integrates ontology lookup services that provide auto-completion for IRIs when authoring RDF triples. This facilitates the reuse of existing ontologies and promotes semantic interoperability.
4. **SPARQL querying support:** The extension introduces the ability to query RDF data using SPARQL. This allows users to retrieve and analyze information from the generated knowledge graph directly within the MetaConfigurator environment.
5. **Interactive knowledge graph visualization:** The RDF triples generated from JSON-LD can be visualized as an interactive graph representation. This visualization enables users to explore entities and relationships within the knowledge graph and understand the semantic structure of the data more easily.
6. **AI-assisted SPARQL and RML authoring from user hints:** Building on MetaConfigurator's existing AI-assisted schema capabilities, this thesis introduces AI-assisted generation of SPARQL queries and RML mappings from user-provided hints. Users can describe their intent in natural language and receive draft SPARQL queries and RML mapping definitions that can be reviewed and refined in the RDF Authoring Panel.

By integrating these capabilities into MetaConfigurator, the platform evolves from a schema-based configuration editor into a tool that supports the full workflow of creating, transforming, querying, and exploring semantically enriched data.

1.4 Related Work

Related work in this area spans several mature but often separate tool families: ontology engineering platforms, transformation tools that convert source data into RDF, schema/metadata modeling environments for structured data, and libraries or engines for querying and validation. These families provide strong support for individual tasks, but integrated workflows that combine guided JSON authoring, semantic transformation, RDF curation, querying, and AI-assisted support in a single user-facing environment are still uncommon.

1.4.1 Ontology Engineering and Knowledge Graph Platforms

Protégé and WebProtégé are widely used for ontology engineering and collaborative ontology curation [CBI; HGN+14]. They provide robust support for class/property modeling and ontology

lifecycle management, but they are not primarily designed as schema-driven editors for end-user JSON instance data.

Apache Jena provides a mature programmatic stack for RDF processing, reasoning, and SPARQL querying [Fou]. GraphDB focuses on RDF storage and query execution in production-oriented knowledge graph settings [Ont]. These systems are strong back-end components, but typically require separate front-end layers for guided data entry and authoring workflows.

1.4.2 Schema and Metadata Modeling for Structured Data

LinkML supports schema-first modeling and can generate artifacts such as JSON-LD and Web Ontology Language (OWL) from a single model [MSU+21]. The Center for Expanded Data Annotation and Retrieval (CEDAR) Workbench supports ontology-assisted metadata authoring for scientific experiments [GOM+19]. Both approaches improve semantic consistency at model level, but they do not directly address an integrated workflow where users iteratively transform arbitrary JSON instance data into RDF using declarative mappings and in-tool graph exploration.

1.4.3 Transformation from Source Data to RDF

Karma, OpenRefine (with RDF extensions), and SPARQL-Generate support transformation from heterogeneous sources to RDF [CG; ISI20; Proa]. These tools are effective for extract, transform, load (ETL)-oriented conversion pipelines, but they focus primarily on mapping and transformation rather than tightly coupled, schema-guided JSON editing and semantic authoring in one interface.

1.4.4 Querying, Validation, and Programmatic RDF Processing

RDF validation and querying standards such as SHACL and SPARQL are well established [PS13; W3C17]. For lightweight experimentation, JSON-LD Playground provides quick inspection of contexts and RDF output [CG23]. RDFLib (RDFLib) and Redland provide low-level programmatic RDF processing [Prob; Proc]. These tools are important ecosystem building blocks but are not, by themselves, unified authoring environments for non-expert users.

1.4.5 Comparison of Representative Tools

Table 1.1 compares representative tools across capabilities relevant to this thesis: RDF authoring, ontology integration, source-to-RDF transformation, RDF storage/query, visualization, and AI assistance.

Table 1.1: Comparison of tools supporting RDF authoring and Semantic Web integration

Tool / Framework	RDF Authoring	Ontology Integration	Source-to-RDF Transform.	RDF Storage / Query	Visualization	AI Assistance
Protégé	✓	✓	–	–	✓	–
WebProtégé	✓	✓	–	–	Limited	–
Apache Jena	✓	✓	Limited	✓	–	–
LinkML	–	✓	Partial	–	–	–
SemTK [Sem]	✓	✓	Partial	✓	✓	–
OpenRefine (RDF)	Limited	–	✓	–	–	–
SPARQL-Generate	–	–	✓	–	–	–
JSON-LD Playground	Limited	–	Limited	–	Limited	–
RDFLib	✓	Limited	–	Partial	–	–
Redland RDF	✓	Limited	–	Partial	–	–

Table 1.2: Rating scale used in Table 1.1

Symbol / Term	**Meaning**
✓	Native, first-class support
Partial	Substantial but scope-limited support (often programmatic rather than end-user focused)
Limited	Basic or indirect support (e.g., plugin-based, narrow subset, or lightweight/read-only capability)
–	Not a core capability

1.4.6 Research Gap and Positioning of This Thesis

Although these tools provide strong support for individual Semantic Web tasks, several limitations remain for practical, user-facing scientific data workflows:

- **Fragmented workflows:** Modeling, mapping, querying, and visualization are often distributed across multiple tools, requiring users to switch between different environments to complete a single data integration task [HBC+21].
- **High technical entry barrier:** Effective use often requires advanced knowledge of OWL, RDF, SPARQL, and mapping languages, which limits accessibility for domain scientists who are not Semantic Web experts [JHC+15].
- **Gap between structured JSON editing and RDF transformation:** In practice, much scientific data is authored and maintained in hierarchical formats such as JSON. While standards such as JSON-LD provide a bridge to linked data [SKL14], moving from conventional JSON structures to RDF still often requires separate mapping artifacts and additional tools (e.g., R2RML/RML) [DSC12; DVC+14].
- **Limited end-to-end support in one UI:** Most tools focus on a single task—such as ontology editing, mapping definition, or querying—rather than supporting the entire workflow from data authoring and transformation to validation, querying, and graph exploration in a unified environment.
- **Limited AI support in semantic authoring workflows:** Despite recent advances in generative AI, assistance for generating artifacts such as SPARQL queries or RML mappings from natural language or user intent is still rarely integrated into Semantic Web authoring tools.

This thesis addresses these gaps by extending MetaConfigurator from a schema-guided data modeling tool into an integrated RDF authoring environment. In particular, it supports a continuous workflow from JSON authoring to JSON-LD/RDF transformation, ontology-aware triple editing, integrated SPARQL querying, graph visualization, and AI-assisted drafting of SPARQL queries and RML mappings.

2 Design and Implementation

2.1 RDF Authoring Panel

The RDF authoring functionality is implemented as a dedicated, separate panel in MetaConfigurator, located in `meta_configurator/src/components/panels/rdf`. This design keeps semantic authoring concerns modular while still integrating with the existing panel configurations, path selection, and data editing flow of MetaConfigurator [NBX+24]. At runtime, the entry component `RdfPanel.vue` validates whether the current data can be treated as JSON-LD and provides immediate warnings for missing `@context`, invalid syntax, and non-JSON-LD input.

Figure 2.1 shows a snapshot of the RDF panel in action, loaded with the JSON example from 1.1. Since the data is provided in JSON format, the panel indicates that it must first be converted to JSON-LD before further processing. The user can proceed in two ways: either by directly supplying a JSON-LD document in the Text View, or by using the JSON-to-JSON-LD conversion tool discussed in Section 2.1.1.

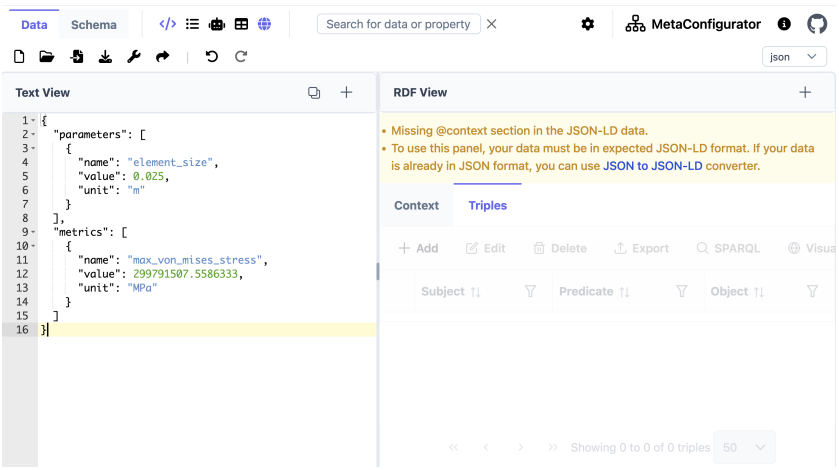

Figure 2.1: A snapshot of MetaConfigurator's RDF panel.

2.1.1 RML Mapping Tool

If the input data is not already expressed as JSON-LD, the RDF panel provides a conversion tool that enables users to transform JSON data into RDF-compatible JSON-LD using RML mappings. RML extends the R2RML mapping language and allows the transformation of heterogeneous data sources such as JSON, CSV, or XML into RDF graphs [DVC+14], as discussed in Section 1.2.6. Figure 2.2 illustrates the dialog used to define such mappings.

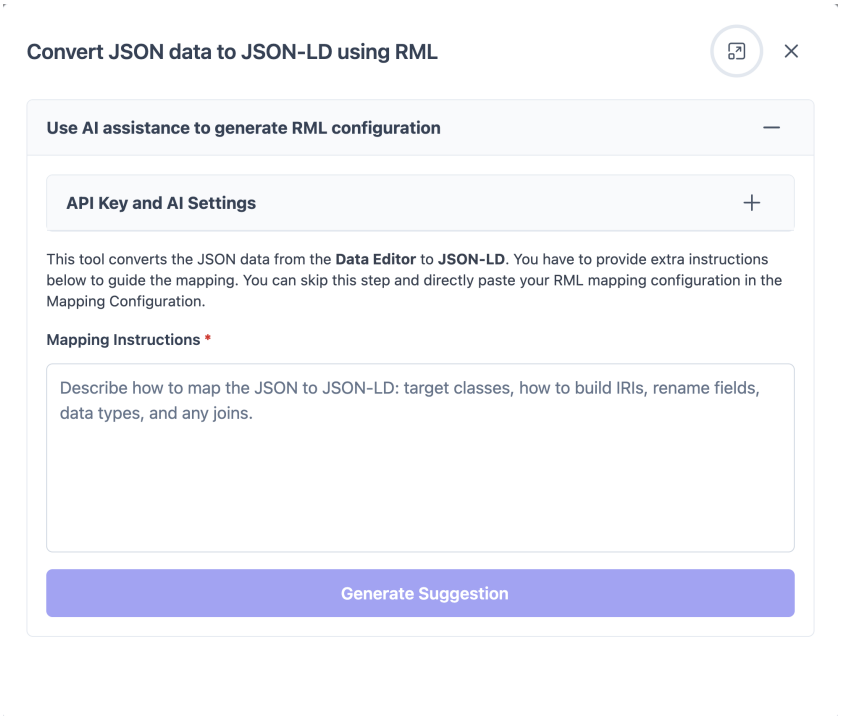

Figure 2.2: The RML mapping dialog for JSON-to-JSON-LD conversion.

The dialog allows users to either provide an existing RML mapping or generate one automatically with AI assistance. In the latter case, the user supplies mapping hints that describe how elements of the input JSON data should correspond to RDF properties or classes. These hints, together with a representative subset of the input data, are sent to an external AI API to generate a candidate mapping.

Internally, the system constructs a structured prompt that includes example mappings and fragments of the input data. This prompt guides the language model to produce a valid RML mapping in Turtle syntax that conforms to the target JSON-LD structure. The prompt constrains the model to return only the mapping definition while adhering to RML rules, thereby improving reliability and syntactic correctness.

```
Convert my JSON Data to JSON-LD such that I have two nodes, each node representing a
    parameter or metric.
```

Listing 2.1: Sample RML hints which describe the mapping from JSON in 1.1 to a minimal JSON-LD in 2.3

The mapping editor is implemented using the CodeMirror component, a widely used web-based code editor that provides syntax highlighting and automatic indentation [Hav23]. Mapping validity is checked by the mapping service through debounced live validation. This allows users to inspect, refine, and validate the generated mapping directly within the interface before applying it to transform the JSON input into JSON-LD.

Users may either provide a RML mapping in Turtle syntax directly or use the AI-assisted generation feature by supplying mapping hints. The resulting mapping is displayed in the CodeMirror editor, where it can be reviewed and edited as needed. Since AI-generated mappings may contain semantic or syntactic errors, user verification is required before execution. Once finalized, the mapping can be applied to convert the input JSON data into JSON-LD, which can then be further managed in the RDF authoring panel.

The examples in this section show that even a minimal hint can already produce a correct mapping for simple targets. In particular, the short natural-language hint shown above can still lead to a valid RML mapping (Listing 2.2) and a corresponding JSON-LD output (Listing 2.3).

However, realistic RML authoring is often more complex and usually requires precise guidance to obtain the intended semantic structure. Therefore, detailed hints such as Listing 2.4 are needed when the target output must follow specific vocabularies, resource patterns, and links between entities. With this level of instruction, the generated mapping can match the expected structure in Listing 1.7 and produce the intended JSON-LD in Listing 1.6.

After providing such hints, the configured AI model generates a mapping that follows the specified structure and semantics. Once the user reviews and confirms the mapping, it can be executed to transform the input JSON data into a semantically enriched JSON-LD representation, as illustrated in Figure 2.3.

```
@prefix rr: <http://www.w3.org/ns/r2rml#> .
@prefix rml: <http://semweb.mmlab.be/ns/rml#> .
@prefix ql: <http://semweb.mmlab.be/ns/ql#> .
@prefix ex: <http://example.com/ns#> .

<#ParameterMapping>
  rml:logicalSource [
    rml:source "Data.json" ;
    rml:referenceFormulation ql:JSONPath ;
    rml:iterator "$.parameters[*]"
  ] ;

  rr:subjectMap [
    rr:template "http://example.com/parameter/{name}" ;
    rr:class ex:Parameter
  ] ;

  rr:predicateObjectMap [
    rr:predicate ex:parameterName ;
    rr:objectMap [ rml:reference "name" ]
  ] ;
  rr:predicateObjectMap [
    rr:predicate ex:parameterValue ;
    rr:objectMap [ rml:reference "value" ]
  ] ;
  rr:predicateObjectMap [
    rr:predicate ex:parameterUnit ;
    rr:objectMap [ rml:reference "unit" ]
  ] .

<#MetricMapping>
  rml:logicalSource [
    rml:source "Data.json" ;
    rml:referenceFormulation ql:JSONPath ;
    rml:iterator "$.metrics[*]"
  ] ;

  rr:subjectMap [
    rr:template "http://example.com/metric/{name}" ;
    rr:class ex:Metric
  ] ;

  rr:predicateObjectMap [
    rr:predicate ex:metricName ;
    rr:objectMap [ rml:reference "name" ]
  ] ;
  rr:predicateObjectMap [
    rr:predicate ex:metricValue ;
    rr:objectMap [ rml:reference "value" ]
  ] ;
  rr:predicateObjectMap [
    rr:predicate ex:metricUnit ;
    rr:objectMap [ rml:reference "unit" ]
  ] .
```

Listing 2.2: RML mapping which converts JSON data in Listing 1.1 to JSON-LD in Listing 2.3

```
{
  "@context": {
    "ex": "http://example.com/ns#"
  },
  "@graph": [
    {
      "@id": "http://example.com/metric/max_von_mises_stress",
      "@type": "ex:Metric",
      "ex:metricName": "max_von_mises_stress",
      "ex:metricUnit": "MPa",
      "ex:metricValue": {
        "@type": "http://www.w3.org/2001/XMLSchema#double",
        "@value": "2.997915075586333E8"
      }
    },
    {
      "@id": "http://example.com/parameter/element_size",
      "@type": "ex:Parameter",
      "ex:parameterName": "element_size",
      "ex:parameterUnit": "m",
      "ex:parameterValue": {
        "@type": "http://www.w3.org/2001/XMLSchema#double",
        "@value": "2.5E-2"
      }
    }
  ]
}
```

Listing 2.3: JSON-LD representation of the simulation result, obtained after applying the minimal RML mapping hint shown above

```
Create one main resource local:run_fenics_simulation of type m4i:Method.
This node links to parameter nodes using m4i:hasParameter and to metric nodes using m4i:investigates.

Iterate over the arrays parameters[*] and metrics[*] separately to create parameter and metric nodes.

For each entry in these arrays:

Create a resource with IRI local:variable_{name} using the "name" field.
Set the type to schema:PropertyValue.
Set rdfs:label to the "name" value.
Set schema:value to the "value".
Map the "unit" field to schema:unitCode as an IRI from the QUDT unit vocabulary using the template
      http://qudt.org/vocab/unit/{unit}.

Connect the method node to parameter nodes using m4i:hasParameter, and to metric nodes using m4i:investigates.

Use these prefixes in the mapping:
m4i: http://w3id.org/nfdi4ing/metadata4ing#
schema: https://schema.org/
rdfs: http://www.w3.org/2000/01/rdf-schema#
unit: http://qudt.org/vocab/unit/
local: https://www.example.org/
```

Listing 2.4: Sample RML hints which describe the mapping from JSON in 1.1 to JSON-LD in 1.6

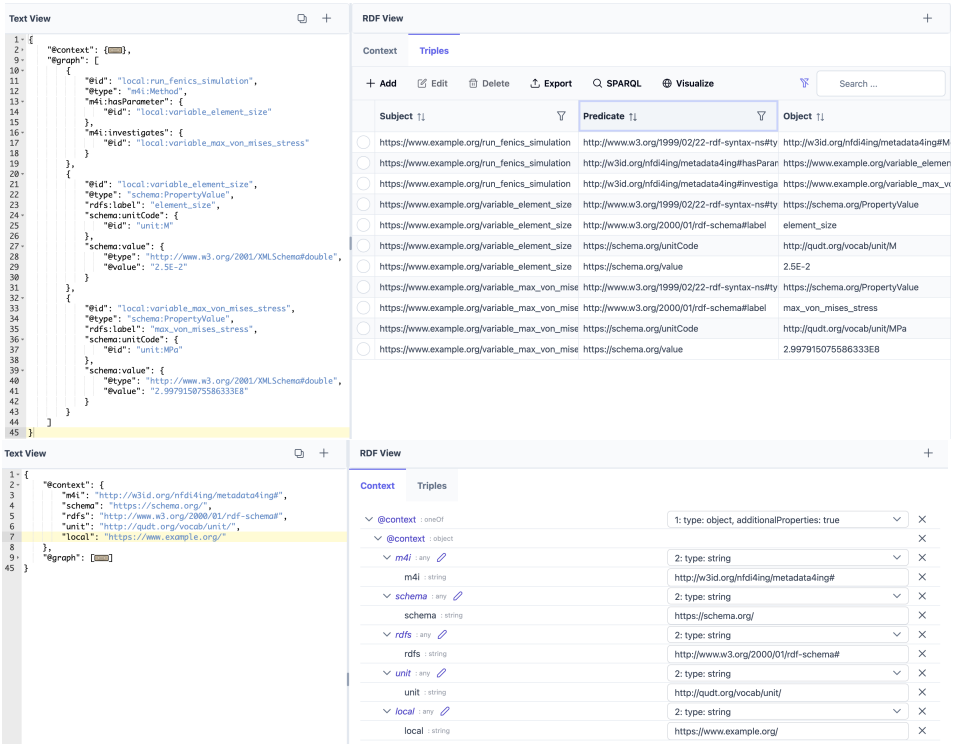

Figure 2.3: JSON-LD output obtained by applying the mapping from Listing 1.7, focused on the Triples and Context tabs, respectively.

2.1.2 Panel Composition and UI Structure

The core editing surface is implemented in `RdfEditorPanel.vue` as a PrimeVue Tabs[2] container. The implementation is primarily organized into two tabs:

- **Context tab**: for editing `@context` definitions.
- **Triples tab**: for browsing and editing RDF triples.

This explicit split mirrors the conceptual separation in JSON-LD between vocabulary declarations and graph assertions [SKL14], and therefore supports both semantic modeling tasks (namespace/prefix handling) and graph authoring tasks (subject-predicate-object manipulation) in one workflow.

When the current document is invalid or not in the expected JSON-LD form, both tabs become visually disabled. This prevents accidental modifications when parsing preconditions are not satisfied.

2.1.3 Context Tab Implementation

The Context tab is implemented by `RdfContextEditorPanel.vue`. It reuses MetaConfigurator's GUI editor and applies a predefined JSON Schema so that `@context` can be edited through structured forms rather than plain text only. The selection updates are synchronized with the text editor view, enabling users to navigate to and manipulate context entries consistently with other panels.

2.1.4 Triples Tab Implementation

The Triples tab is implemented by `RdfTripleEditorPanel.vue`. It shows triples in a PrimeVue `DataTable` [3] with:

- paging, sorting, and column/global filtering,
- single-row selection synchronized with document path selection in the text editor,
- actions for add, edit, delete, export, SPARQL, and visualization.

Triple editing is handled through `TripleDetailsDialog.vue`, which allows controlled entry of subject, predicate, and object terms, including typed literals with XML Schema datatypes. Changes are executed through `TripleEditorService` and propagated to the central RDF store. Figure 2.4 shows the edit modal for triple editing.

[2] `https://primevue.org/tabs/`
[3] `https://primevue.org/datatable/`

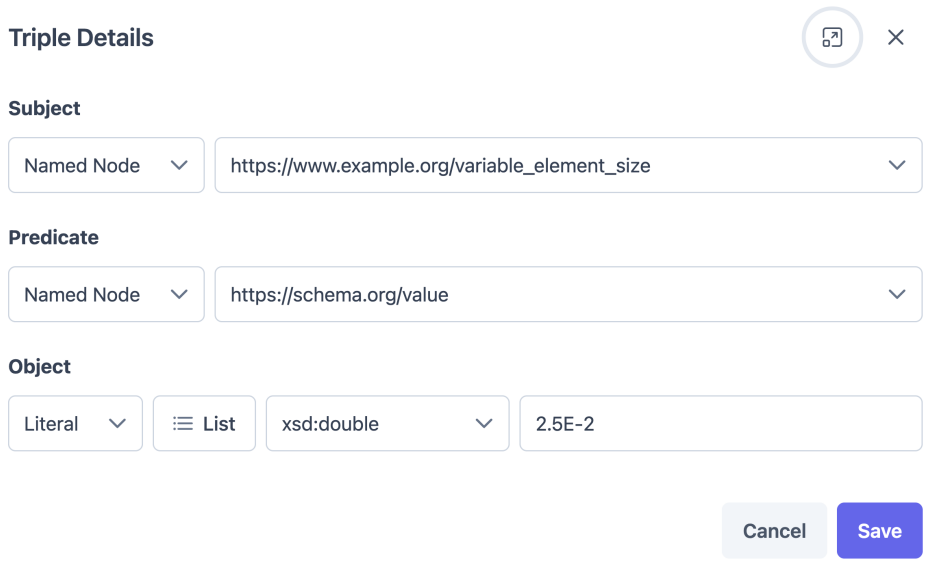

Figure 2.4: Edit modal for triple editing.

Data changes occur from two sources: the Text View (raw JSON-LD data) or the RDF Panel. When a change is made in the Text View, `rdfStoreManager.ts` rebuilds the RDF graph and updates the RDF Panel (Figure 2.5).

Conversely, when a triple is edited, removed, or added in Triples tab, `TripleEditorService` validates and applies the change through `rdfStoreManager.ts`. If successful, `jsonLdNodeManager.ts` rebuilds JSON-LD from the updated store and writes it back to the Data Editor state, which updates the Text View accordingly (Figure 2.6). Change in the Context tab is propagated in the same way as changes to the GUI View, since underlying data is still treated as JSON data and processed by the same synchronization workflow.

2.1.5 RDF Store and Data Synchronization

The internal semantic state is managed by `rdfStoreManager.ts`, which uses `rdflib.js` to parse and serialize data [lin26]. On each source-data update, the manager:

1. resolves `@context` data into namespace mappings,
2. parses JSON-LD into an indexed RDF graph and derives a reactive list of Statements,
3. exposes reactive Statements, warnings, and errors to UI components.

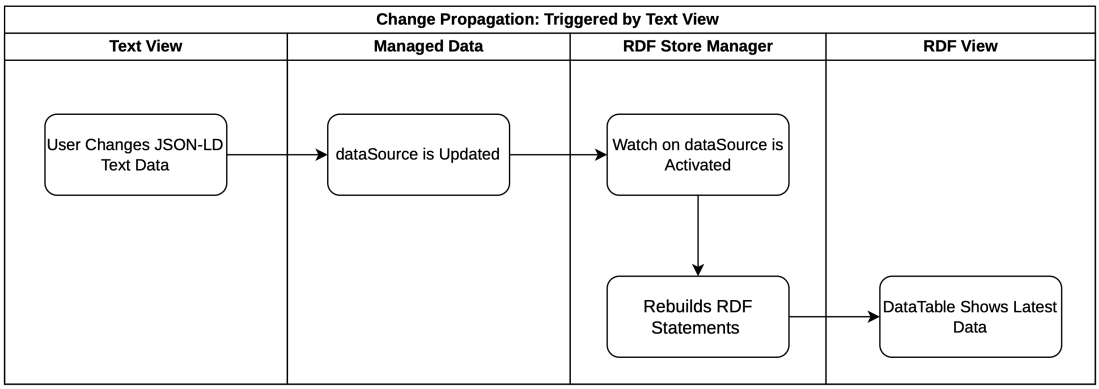

Figure 2.5: Synchronization process illustrating how modifications made in the Text View are propagated and reflected in the RDF View.

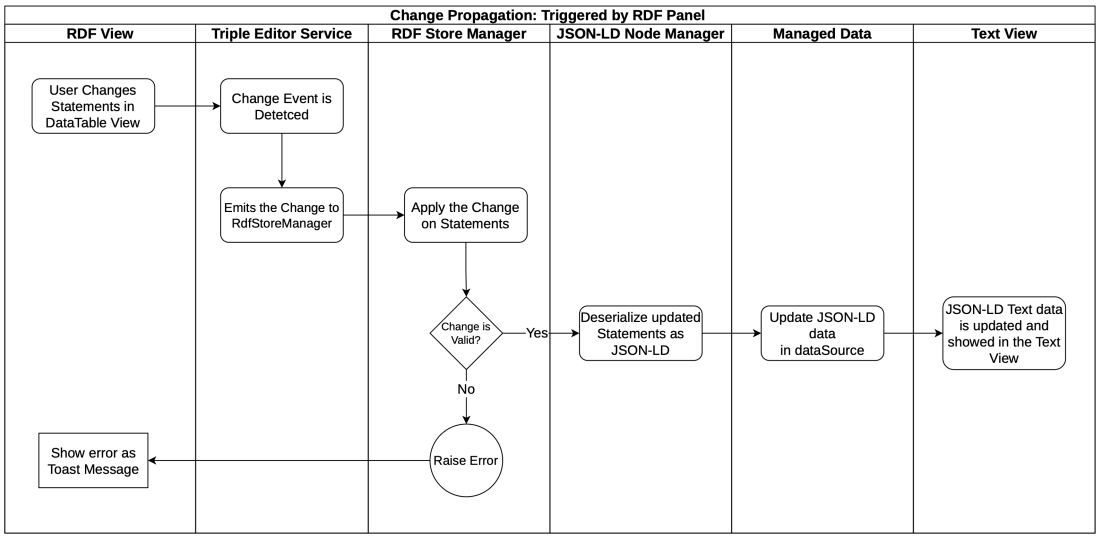

Figure 2.6: Workflow showing how changes originating in the RDF View are transferred back and represented in the Text View.

Definition 2.1 (RDFLib Statement)
In this implementation, an RDFLib Statement is the runtime representation of one RDF triple of the form (s, p, o)*, where* s *(subject),* p *(predicate), and* o *(object) are RDF terms. In* `rdflib.js`*, these terms are represented as* `NamedNode`, `BlankNode`*, and* `Literal` *objects, and each triple is stored as a Statement. Briefly, a* `NamedNode` *represents an identified resource via an IRI (e.g., ontology classes, predicates, or linked entities), a* `BlankNode` *represents an unnamed local resource without a global IRI, and a* `Literal` *represents a concrete value such as text, number, or date (optionally with datatype or language tag). A collection of such Statements forms the in-memory graph used by the panel [lin26].*

This definition is directly reflected in the implementation workflow: the internal store parses JSON-LD into a reactive list of Statement objects, the triples table renders this list, and editing actions create or modify individual statements for add/edit/delete operations. Statement-level matching (subject, predicate, object) is used to synchronize table selection with text-editor paths.

To keep the JSON editor and RDF authoring panel consistent, the implementation maps RDF statements back into JSON-LD data and computes path correspondences between selected triples and document positions. This enables bidirectional synchronization: selecting a triple can focus the matching JSON path, and raw JSON data changes trigger store rebuilds.

2.1.6 Linked Selection Between Text View and RDF View

Linked selection between the Text View and the RDF View enables coordinated highlighting of corresponding elements across both representations. When a user selects a section of the JSON-LD document, the corresponding RDF statement is identified and highlighted in the DataTable, and vice versa. This maintains a consistent visual selection state across both views. Figure 2.7 illustrates this behavior. Algorithms 2.1 and 2.2 describe the implementation of linked selection in both directions.

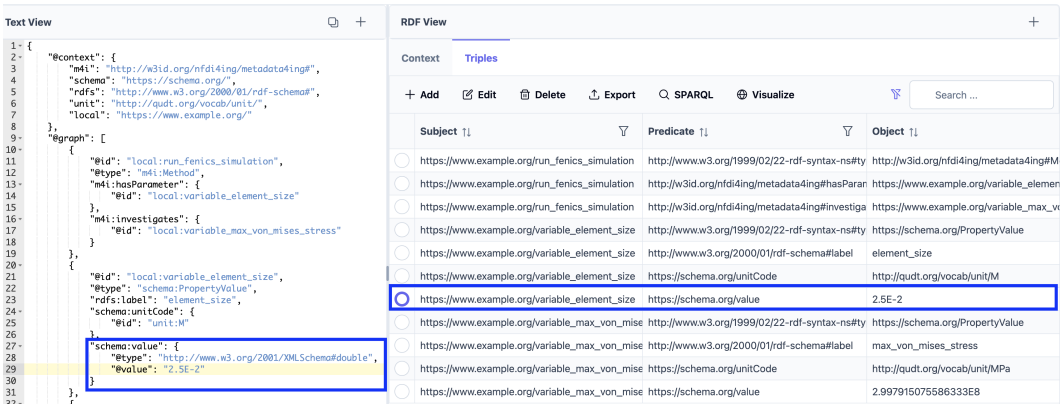

Figure 2.7: Linked selection between Text View and RDF View. The highlighted JSON fragment corresponds to the selected triple in the DataTable, and vice versa.

Algorithmus 2.1 Linked selection from Text View to RDF View

1: **Input:** current JSON path selection, full JSON-LD document, current DataTable rows
2: Read the selected JSON path from the editor state
3: Extract subject (`@id`), predicate, and object at the selected path
4: Build a minimal JSON-LD fragment containing:
   a) original `@context`
   b) one `@graph` node with extracted subject, predicate, and object
5: Parse the minimal fragment with RDFLib into a temporary graph
6: **if** the temporary graph contains exactly one statement **then**
7: Compare that statement against each DataTable statement by subject, predicate, and object equality
8: **if** a matching statement is found **then**
9: Determine its row index in the DataTable
10: Set DataTable selection to the matching row
11: Scroll/focus the table to the selected row

Algorithmus 2.2 Linked selection from RDF View to Text View

Input: selected DataTable statement (s, p, o), current JSON-LD document
Read subject s, predicate p, and object o from the selected table row
Find candidate node in `@graph` whose `@id` matches s
if no candidate node is found **then**
    return without navigation
Find predicate entry in the candidate node that matches p
    (consider both prefixed terms and full IRIs as equivalent)
if no matching predicate entry is found **then**
    return without navigation
Match object o within the predicate value
    (as `@id`, `@value`, or scalar value)
if an object match is found **then**
    Compute the corresponding JSON path
    Move editor cursor/focus to that JSON path

2.2 Query with SPARQL

The SPARQL query functionality is implemented in `SparqlEditor.vue` as a dialog with three tabs: **Query View**, **Result View**, and **Visualization**. During implementation, RDFLib query capabilities were evaluated first, but they proved limited for the required in-browser query workflow and did not provide full support for current SPARQL 1.2 draft specifications. Query execution is therefore performed in-browser using Comunica's `QueryEngine` [Ass26a]. According to Comunica's official documentation, the engine supports SPARQL 1.2 draft query-related specifications [Ass26b].

In the **Query View**, users can write and validate queries in a CodeMirror editor [Hav23] and execute them against the current in-memory RDF graph. Query syntax is checked through debounced live validation using the SparqlJS parser, which provides immediate feedback while editing [Vc26]. The same view also provides AI-assisted SPARQL generation. Similar to the AI-assisted RML workflow described in Section 2.1.1, user hints are combined with prompt-engineering instructions and a representative subset of the current JSON-LD data, then sent to the configured Large Language Model (LLM) to generate a draft SPARQL query. Figure 2.8 shows the dialog with an AI-assisted query suggestion in the Query View.

2.2.1 Showcase: AI-Assisted SPARQL Query Generation

To demonstrate the AI-assisted workflow, we assume that the simulation JSON from Listing 1.1 has been extended with additional `element_size` and `max_von_mises_stress` value pairs (e.g., from multiple simulation runs), and then transformed into JSON-LD using the mapping workflow

described earlier. In this extended dataset, metric measurements are available for multiple element sizes, for example `0.003125`, `0.00625`, `0.0125`, `0.05` and `0.1`.

In this scenario, the user provides a natural-language hint in the Query View to request a query that returns paired values of `element_size` and `max_von_mises_stress`. An example hint is shown in Listing 2.5. The hint and a subset of the current JSON-LD graph are sent to the AI API, which proposes a draft query that may still contain syntactic or semantic errors. Therefore, the generated query must be reviewed and, if necessary, refined by the user before execution. A representative generated query is shown in Listing 2.6.

```
Return pairs of element_size and max_von_mises_stress.
For each run, show the numeric element size value and the numeric stress value.
Sort by element_size ascending.
```

Listing 2.5: Example user hint for AI-assisted SPARQL generation

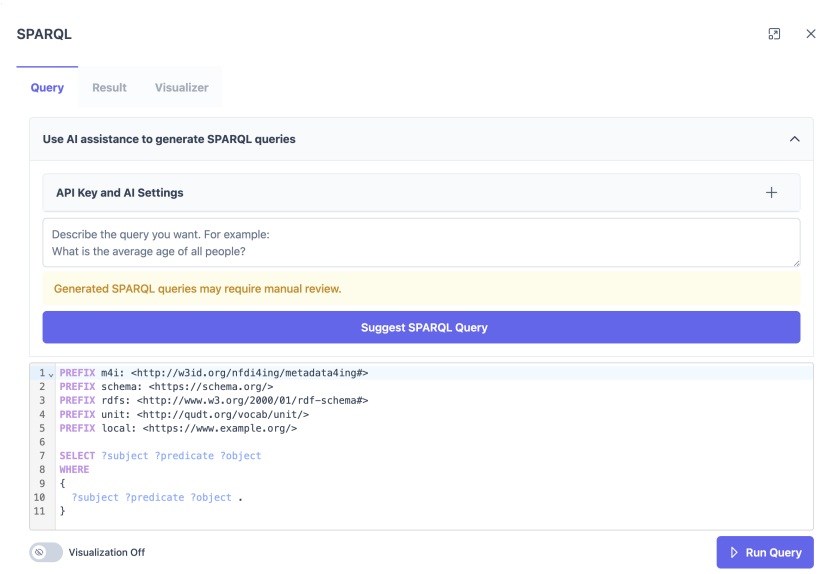

Figure 2.8: SPARQL query dialog, with AI-assisted query generation.

This showcase illustrates the intended interaction pattern: AI is used to accelerate query authoring, while final semantic and syntactic validation remains under user control before execution. Human-in-the-loop review is therefore essential in both AI-generated RML mappings and AI-generated SPARQL queries, and users can inspect and refine both results before applying or executing them.

```
PREFIX m4i: <http://w3id.org/nfdi4ing/metadata4ing#>
PREFIX schema: <https://schema.org/>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
PREFIX unit: <http://qudt.org/vocab/unit/>
PREFIX local: <https://www.example.org/>

SELECT ?element_size ?max_von_mises_stress WHERE {
  ?method a m4i:Method .
  ?method m4i:hasParameter ?element_size_param .
  ?element_size_param a schema:PropertyValue ;
                      schema:value ?element_size .
  ?method m4i:investigates ?stress_param .
  ?stress_param a schema:PropertyValue ;
                schema:value ?max_von_mises_stress .
} ORDER BY ASC(?element_size)
```

Listing 2.6: AI-suggested SPARQL query for `element_size` vs. `max_von_mises_stress` pairs

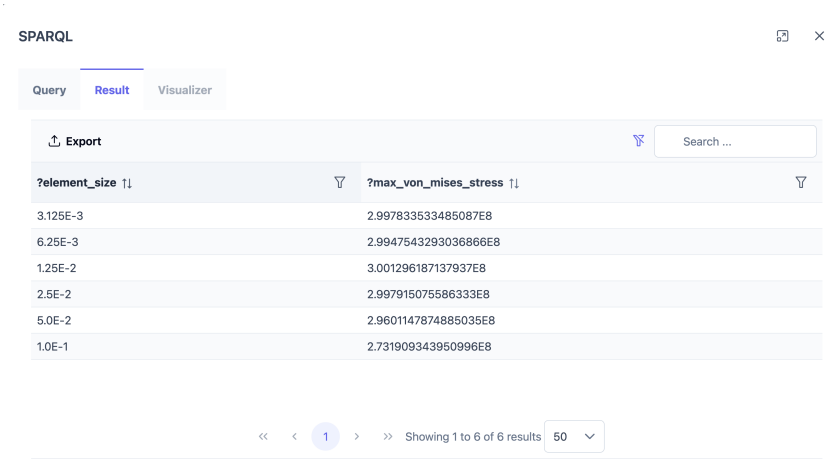

Figure 2.9: Result after running SPARQL query in Figure 2.8.

The dialog supports two execution modes controlled by a toggle: **Visualization Off** and **Visualization On**. With **Visualization Off**, standard SELECT-style querying is used and results are shown in the **Result View**. There, the PrimeVue DataTable columns are generated dynamically from the returned variables, and export is provided as CSV. Figure 2.10 shows a plot which is generated from the CSV export of the query result, visualizing the relationship between element size and maximum von Mises stress.

With **Visualization On**, the query mode switches to CONSTRUCT-based graph output; the same result table is still populated, but export options target RDF serializations (e.g., Turtle, N-Triples, RDF/XML) for graph-oriented workflows.

The **Visualization** tab renders the query result graph in read-only mode. Its interaction model and rendering details are discussed in the Section 2.3.

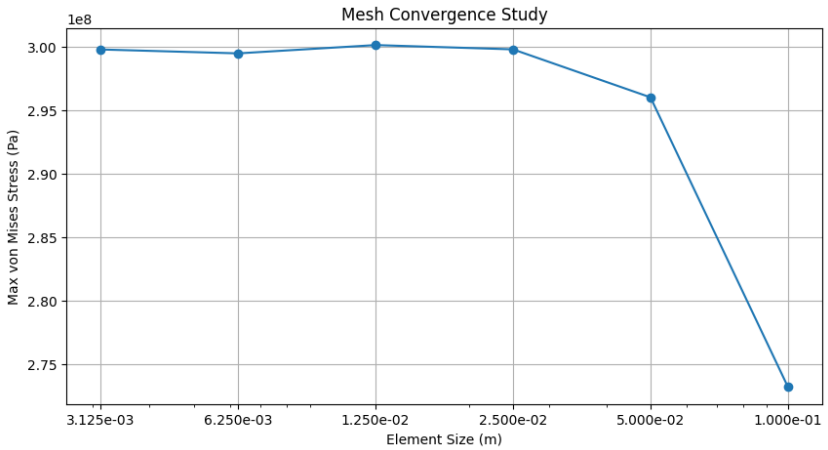

Figure 2.10: Data exported as CSV could be visualized as a plot of `element_size` vs. `max_von_mises_stress`, revealing trends in the simulation results.

2.3 Knowledge Graph Visualization

Knowledge graph rendering is implemented in `RdfVisualizer.vue` using Cytoscape.js [FLH+16], with the COSE-Bilkent layout extension for force-directed graph placement. The visualization presents resources as nodes and semantic relations as directed edges, while preserving readable labels through namespace-aware prefix shortening.

The view provides interactive graph navigation features including zoom in/out, fit-to-view, and zoom-to-selected-node actions. For performance and usability, large graphs are detected before rendering and users are prompted to continue or cancel. In addition, graph export is supported as an image, and an optional animation mode can recompute the layout to improve readability after edits.

Node inspection and editing support are integrated through `RdfVisualizerPropertiesView.vue`. When a node is selected, a side panel shows the node identifier and its properties, including linkable IRIs. The same panel exposes lightweight authoring actions (add/edit/delete property, rename/delete node, add node), so users can inspect and refine graph content directly from the visualization workflow.

The visualization tab can be opened in two ways. First, it can be opened directly from the Triples tab of the RDF panel. In this mode, the visualization is editable, and user interactions are propagated back to the main JSON-LD data and RDF store. Second, it can be opened from the SPARQL dialog when the **Visualization On** toggle is enabled. In that case, the graph is built from the user's SPARQL CONSTRUCT result and is shown in read-only mode, because returned triples are not guaranteed to map one-to-one to statements in the internal RDF store.

For editable visualization workflows, undo and redo actions are also integrated in the graph toolbar. This allows users to safely revise node/property edits while keeping visualization and underlying data synchronized.

To improve navigation in larger graphs, the visualization also provides node search via PrimeVue AutoComplete[4] component: users receive suggestions of available nodes while typing, can quickly select the intended node, and the view automatically focuses on it.

The visualization also supports clickable hyperlinks for semantic terms. Predicate labels can open their ontology definitions; for example, `m4i:hasParameter` links to predicate definition in the ontology[5]. For object values, `NamedNode` are handled context-sensitively: if the referenced node exists in the current graph, clicking it focuses that node in the visualization; if it points to an external resource, clicking opens the external definition (e.g., `unit:M` linking to `https://qudt.org/vocab/unit/M`).

[4] `https://primevue.org/autocomplete/`

[5] `https://nfdi4ing.pages.rwth-aachen.de/metadata4ing/metadata4ing/index.html#hasParameter`

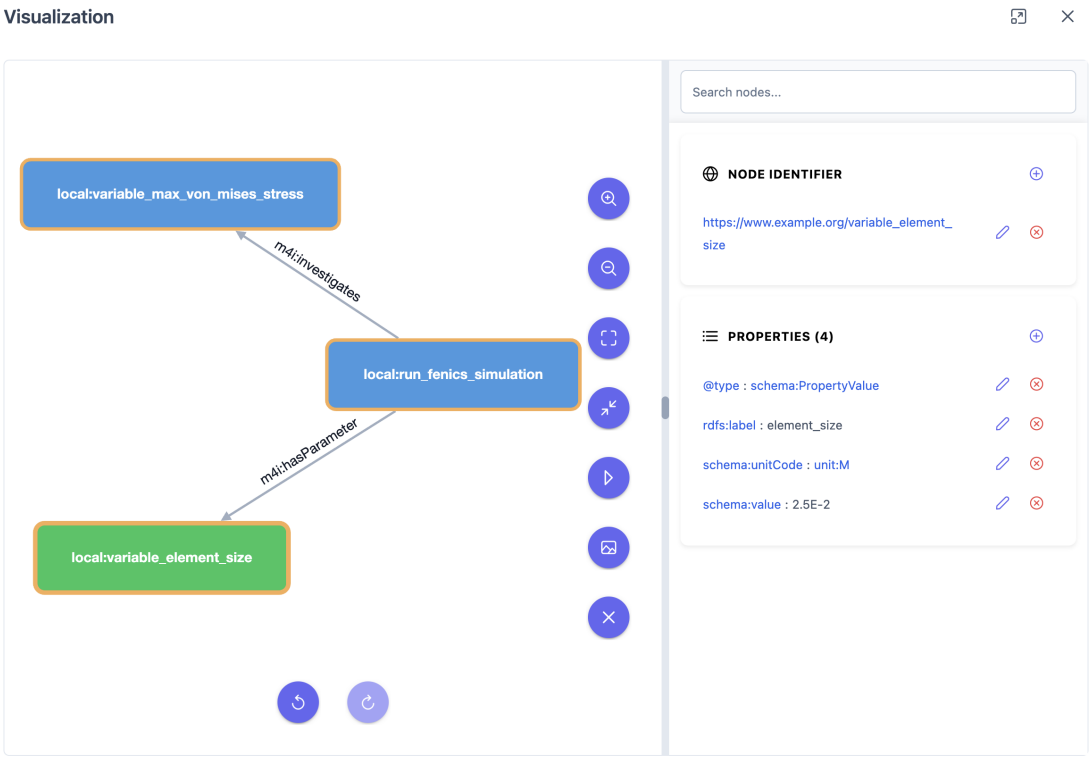

Figure 2.11: Knowledge graph visualization of the JSON-LD data from Listing 1.6, rendered in the Visualization dialog.

3 Application

This chapter demonstrates the end-to-end application of the thesis workflow to Metal-Organic Framework (MOF) synthesis data. Structured experiment records are modeled in JSON, transformed into semantically enriched RDF/JSON-LD using RML, and then prepared for ontology-aware querying and graph-based analysis.

In line with the motivation in Chapters 1 and 2, the main idea is to preserve laboratory protocol detail (materials, actions, and conditions) while adding formal semantics so that the same data can be interpreted consistently across tools and datasets.

3.1 JSON Structure of the MOF Synthesis Dataset

MOF synthesis refers to the stepwise preparation of crystalline coordination materials from metal precursors and organic linkers under controlled laboratory conditions. The synthesis process is not only defined by the final product, but also by the full protocol context (reagents, roles, action sequence, and process conditions), which should be represented in structured, machine-readable form. [NEB+26]

Listings 3.1 and 3.2 show a representative subset of this dataset. Each top-level entry under `Synthesis[*]` encodes one complete run (e.g., `S-1`, `S-2`) with four core components: hardware setup, run metadata, ordered procedure phases (Prep/Reaction/Workup), and reagent inventory with functional roles. This representation provides the structural basis required for the semantic transformation step discussed in the next section.

3.2 RML Mapping for Semantic Enrichment

The RML mappings below in Listings 3.3 and Listings 3.4 show how the JSON structures defined in Listings 3.1 and Listings 3.2 are transformed into typed RDF resources, as described in Section 1.2.6. In MetaConfigurator, this is the operational step that bridges conventional JSON input and the RDF/JSON-LD authoring and query workflow (Section 2.1.1).

```
{
  "Synthesis": [
    {
      "Hardware": { "Component": [{ "_id": "S-1", "_type": "glass vial" }] },
      "Metadata": { "_description": "S-1", "_product": "MIL-88B (Fe)" },
      "Procedure": {
        "Prep": {
          "Step": [
            { "_vessel": "S-1", "$xml_type": "Add", "_amount": { "Unit": "millimole", "Value": 0.4},
                 "_reagent": "FeCl3", "_order": 0 },
            { "_vessel": "S-1", "$xml_type": "Add", "_amount": { "Unit": "millimole", "Value": 0.4},
                 "_reagent": "Benzene-1,4-dicarboxylic acid", "_order": 1 },
            { "_vessel": "S-1", "$xml_type": "Add", "_amount": { "Unit": "millilitre", "Value": 4}, "_reagent":
                 "DMF", "_order": 2 }
          ]
        },
        "Reaction": {
          "Step": [
            { "_vessel": "S-1", "$xml_type": "Sonicate", "_time": { "Value": 30.0, "Unit": "minute"}, "_order":
                 0 },
            { "_comment": "In: oven", "_vessel": "S-1", "$xml_type": "HeatChill", "_temp": { "Unit": "celsius",
                 "Value": 120.0}, "_time": { "Value": 1, "Unit": "day"}, "_order": 1 }
          ]
        },
        "Workup": {
          "Step": [
            { "_vessel": "S-1", "$xml_type": "WashSolid", "_solvent": "EtOH", "_order": 0 },
            { "_vessel": "S-1", "$xml_type": "Dry", "_temp": { "Unit": "celsius", "Value": 120.0}, "_time": {
                 "Value": 1, "Unit": "day"}, "_pressure": { "Unit": "pascal", "Value": 0}, "_order": 1 }
          ]
        }
      },
      "Reagents": {
        "Reagent": [
          { "_id": "FeCl3", "_name": "FeCl3", "_role": "substrate" },
          { "_id": "Benzene-1%252C4-dicarboxylic%2520acid", "_name": "Benzene-1,4-dicarboxylic acid", "_role":
               "ligand" },
          { "_id": "DMF", "_name": "DMF", "_role": "solvent" },
          { "_id": "EtOH", "_name": "EtOH", "_role": "solvent" }
        ]
      }
    }
  ]
}
```


Listing 3.1: Representative JSON excerpt from the MOF synthesis dataset for S-1

```
{
  "Synthesis": [
    {
      "Hardware": { "Component": [{ "_id": "S-2", "_type": "glass vial" }] },
      "Metadata": { "_description": "S-2", "_product": "MIL-88B+MIL101 (mix phases)" },
      "Procedure": {
        "Prep": {
          "Step": [
            { "_vessel": "S-2", "$xml_type": "Add", "_amount": { "Unit": "millimole", "Value": 0.4},
                 "_reagent": "FeCl3\\u00B76H2O", "_order": 0 },
            { "_vessel": "S-2", "$xml_type": "Add", "_amount": { "Unit": "millimole", "Value": 0.4},
                 "_reagent": "Benzene-1,4-dicarboxylic acid", "_order": 1 },
            { "_vessel": "S-2", "$xml_type": "Add", "_amount": { "Unit": "millilitre", "Value": 4}, "_reagent":
                 "DMF", "_order": 2 }
          ]
        },
        "Reaction": {
          "Step": [
            { "_vessel": "S-2", "$xml_type": "Sonicate", "_time": { "Value": 30.0, "Unit": "minute"}, "_order":
                 0 },
            { "_comment": "In: oven", "_vessel": "S-2", "$xml_type": "HeatChill", "_temp": { "Unit": "celsius",
                 "Value": 120.0}, "_time": { "Value": 1, "Unit": "day"}, "_order": 1 }
          ]
        },
        "Workup": {
          "Step": [
            { "_vessel": "S-2", "$xml_type": "WashSolid", "_solvent": "EtOH", "_order": 0 },
            { "_vessel": "S-2", "$xml_type": "Dry", "_temp": { "Unit": "celsius", "Value": 120.0}, "_time": {
                 "Value": 1, "Unit": "day"}, "_pressure": { "Unit": "pascal", "Value": 0}, "_order": 1 }
          ]
        }
      },
      "Reagents": {
        "Reagent": [
          { "_id": "FeCl3%25C2%25B76H2O", "_name": "FeCl3\\u00B76H2O", "_role": "substrate" },
          { "_id": "Benzene-1%252C4-dicarboxylic%2520acid", "_name": "Benzene-1,4-dicarboxylic acid", "_role":
               "ligand" },
          { "_id": "DMF", "_name": "DMF", "_role": "solvent" },
          { "_id": "EtOH", "_name": "EtOH", "_role": "solvent" }
        ]
      }
    }
  ]
}
```


Listing 3.2: Representative JSON excerpt from the MOF synthesis dataset for S-2

```
@prefix rr: <http://www.w3.org/ns/r2rml#> .
@prefix rml: <http://semweb.mmlab.be/ns/rml#> .
@prefix ql: <http://semweb.mmlab.be/ns/ql#> .
@prefix ex: <http://example.org/> .
@prefix chmo: <http://purl.obolibrary.org/obo/CHMO_> .
@prefix obo: <http://purl.obolibrary.org/obo/> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .

<#SynthesisSource> rml:source "Data.json" ; rml:referenceFormulation ql:JSONPath ; rml:iterator "
     $.Synthesis[*]" .

<#SynthesisTriplesMap>
  rml:logicalSource <#SynthesisSource> ;
  rr:subjectMap [ rr:template "http://example.org/synthesis/{Hardware.Component[0]._id}" ; rr:class
       chmo:0000410 ] ;
  rr:predicateObjectMap [ rr:predicate ex:componentID ; rr:objectMap [ rml:reference "
       Hardware.Component[0]._id" ] ] ;
  rr:predicateObjectMap [ rr:predicate ex:componentType ; rr:objectMap [ rml:reference "
       Hardware.Component[0]._type" ] ] ;
  rr:predicateObjectMap [ rr:predicate rdfs:label ; rr:objectMap [ rml:reference "Metadata._description" ] ] ;
  rr:predicateObjectMap [ rr:predicate chmo:hasProduct ; rr:objectMap [ rml:reference "Metadata._product" ] ] .
```

Listing 3.3: Part of RML mapping focused on synthesis-level hardware and metadata

```
<#ReagentSource> rml:source "Data.json" ; rml:referenceFormulation ql:JSONPath ; rml:iterator "
     $.Synthesis[*].Reagents.Reagent[*]" .

<#ReagentTriplesMap>
  rml:logicalSource <#ReagentSource> ;
  rr:subjectMap [ rr:template "http://example.org/reagent/{_id}" ; rr:class obo:CHEBI_24431 ] ;
  rr:predicateObjectMap [ rr:predicate rdfs:label ; rr:objectMap [ rml:reference "_name" ] ] ;
  rr:predicateObjectMap [ rr:predicate ex:role ; rr:objectMap [ rml:reference "_role" ] ] .

<#PrepStepSource> rml:source "Data.json" ; rml:referenceFormulation ql:JSONPath ; rml:iterator "
     $.Synthesis[*].Procedure.Prep.Step[*]" .

<#PrepStepTriplesMap>
  rml:logicalSource <#PrepStepSource> ;
  rr:subjectMap [ rr:template "http://example.org/step/prep/{_vessel}-{_order}" ; rr:class obo:OBI_0000070 ] ;
  rr:predicateObjectMap [ rr:predicate ex:phase ; rr:objectMap [ rr:constant "Prep" ] ] ;
  rr:predicateObjectMap [ rr:predicate ex:action ; rr:objectMap [ rml:reference "$xml_type" ] ] ;
  rr:predicateObjectMap [ rr:predicate ex:amountValue ; rr:objectMap [ rml:reference "_amount.Value" ] ] ;
  rr:predicateObjectMap [ rr:predicate ex:amountUnit ; rr:objectMap [ rml:reference "_amount.Unit" ] ] .
```

Listing 3.4: Part of RML mapping focused on reagents and preparation steps

3.2.1 How Ontology Classes Add Semantics to JSON Data

A central semantic effect of the mapping is ontology-based typing in `rr:subjectMap`. These class assignments transform plain JSON objects into explicit domain entities:

- **`chmo:0000410`** types each synthesis run as a chemistry synthesis entity.
- **`obo:CHEBI_24431`** types each reagent node as a chemical entity.
- **`obo:OBI_0000070`** types each procedure step as an experimental/process step entity.

This is important because JSON keys such as `Synthesis`, `Reagent`, and `Step` are structural labels only. After RML transformation, they become ontology-grounded RDF nodes that can be queried by meaning and reused across datasets. This is the semantic lifting described in Sections 1.2.5 and 1.2.6.

In practice, these typed resources directly support the Chapter 2 workflow: they can be inspected in the RDF panel, queried with SPARQL, and explored through graph visualization (Sections 2.2 and 2.3).

4 Conclusion and Outlook

This thesis extends MetaConfigurator with an integrated RDF authoring workflow that bridges schema-driven JSON editing and Semantic Web technologies. The implementation contributes four main capabilities: transformation from JSON to JSON-LD via RML, direct editing of semantic structures through context and triple views, in-browser SPARQL querying with validation support, and interactive graph visualization. In addition, AI-assisted generation of RML mappings and SPARQL queries reduces authoring effort while preserving user control through mandatory review.

The resulting system supports an end-to-end process in a single interface: users can model and validate structured data, enrich it semantically, inspect and edit generated triples, run semantic queries, and explore graph structures without switching tools. This addresses a central gap in existing workflows, where authoring, mapping, querying, and visualization are often fragmented across separate environments.

Several limitations remain. First, AI-generated artifacts can still contain semantic mistakes and therefore require careful user verification. Second, scalability of in-browser graph processing and visualization is bounded by client resources for larger datasets. Third, ontology integration is currently focused on assisted lookup and authoring support rather than full reasoning-based guidance.

Future work should focus on three directions. (1) Robust validation pipelines: integrate SHACL-based validation directly into the authoring workflow to provide machine-checkable semantic constraints before export or publication. (2) Better scalability: add progressive loading, result-size controls, and graph summarization strategies for larger knowledge graphs. (3) Deeper assistance: improve AI-supported authoring with constraint-aware prompting and automated quality checks that detect likely mapping/query errors before execution.

Overall, the presented extension demonstrates that schema-guided data engineering and Semantic Web authoring can be effectively combined in one user-facing environment, improving accessibility and interoperability for scientific data workflows.

Bibliography

[Ass26a] C. Association. *@comunica/query-sparql*. 2026. URL: https://www.npmjs.com/package/@comunica/query-sparql (cit. on p. 46).

[Ass26b] C. Association. *Comunica: Supported specifications*. 2026. URL: https://comunica.dev/docs/query/advanced/specifications/ (cit. on p. 46).

[BHL01] T. Berners-Lee, J. Hendler, O. Lassila. "The Semantic Web". In: *Sci. Amer.* 284.5 (2001), pp. 28–37 (cit. on p. 18).

[Bra17] T. Bray. *The JavaScript Object Notation (JSON) Data Interchange Format*. RFC 8259. 2017. URL: https://www.rfc-editor.org/rfc/rfc8259 (cit. on p. 15).

[CBI] S. CBIR. *Protégé*. URL: https://protege.stanford.edu (cit. on p. 30).

[CG] S.-G. CG. *SPARQL-Generate*. URL: https://github.com/SPARQL-Generate (cit. on p. 31).

[CG23] J.-L. CG. *JSON-LD Playground*. 2023 (cit. on p. 31).

[DSC12] S. Das, S. Sundara, R. Cyganiak. *R2RML: RDB to RDF Mapping Language*. https://www.w3.org/TR/r2rml/. W3C Recommendation. 2012 (cit. on p. 33).

[DVC+14] A. Dimou, M. Vander Sande, P. Colpaert, R. Verborgh, E. Mannens, R. Van de Walle. "RML: A Generic Language for Integrated RDF Mappings of Heterogeneous Data". In: *7th Workshop on Linked Data on the Web*. 2014 (cit. on pp. 22, 33, 36).

[FLH+16] M. Franz, C. T. Lopes, G. Huck, Y. Dong, O. Sumer, G. D. Bader. "Cytoscape.js: A graph theory library for visualisation and analysis". In: *Bioinformatics* 32.2 (2016), pp. 309–311. DOI: 10.1093/bioinformatics/btv557. URL: https://js.cytoscape.org/ (cit. on p. 50).

[Fou] A. S. Foundation. *Apache Jena*. URL: https://jena.apache.org/ (cit. on p. 31).

[GOM+19] R. S. Gonçalves, M. J. O'Connor, M. Martínez-Romero, et al. "The CEDAR Workbench: An Ontology-Assisted Environment for Authoring Metadata that Describe Scientific Experiments". In: *arXiv preprint arXiv:1905.06480* (2019) (cit. on p. 31).

[Gru93] T. R. Gruber. "A Translation Approach to Portable Ontology Specifications". In: *Knowl. Acquis.* 5.2 (1993), pp. 199–220 (cit. on p. 18).

[Gua98] N. Guarino. "Formal Ontology and Information Systems". In: *Formal Ontology in Information Systems*. IOS Press, 1998, pp. 3–15 (cit. on p. 18).

[Hav23] M. Haverbeke. *CodeMirror: In-browser Code Editor*. 2023. URL: https://codemirror.net/ (cit. on pp. 37, 46).

[HBC+21] A. Hogan, E. Blomqvist, M. Cochez, C. d'Amato, G. de Melo, C. Gutierrez, S. Kirrane, J. E. Labra Gayo, R. Navigli, S. Neumaier, A.-C. N. Ngomo, A. Polleres, S. M. Rashid, A. Rula, L. Schmelzeisen, J. Sequeda, S. Staab, A. Zimmermann. "Knowledge Graphs". In: *ACM Computing Surveys* 54.4 (2021), pp. 1–37. DOI: 10.1145/3447772 (cit. on p. 33).

[HGN+14] M. Horridge, R. S. Gonçalves, C. Nyulas, T. Tudorache, M. A. Musen. "WebProtégé: A Collaborative Web-Based Platform for Editing Biomedical Ontologies". In: *Bioinformatics* (2014) (cit. on p. 30).

[ISI20] U. ISI. *Karma: A Data Integration Tool for Mapping Structured Data to RDF*. 2020 (cit. on p. 31).

[JHC+15] K. Janowicz, A. Haller, S. J. D. Cox, D. Le Phuoc, M. Lefrançois. "Why the Data Train Needs Semantic Rails". In: *AI Magazine* 36.1 (2015), pp. 5–14. DOI: 10.1609/aimag.v36i1.2568 (cit. on p. 33).

[lin26] linkeddata. *rdflib.js Documentation*. 2026. URL: https://linkeddata.github.io/rdflib.js/ (cit. on pp. 42, 44).

[MSU+21] S. Moxon, H. Solbrig, D. R. Unni, et al. "Linked Data Modeling Language (LinkML): A General-Purpose Data Modeling Framework". In: *Semantic Web J.* (2021) (cit. on p. 31).

[NBX+24] F. Neubauer, P. Bredl, M. Xu, K. Patel, J. Pleiss, B. Uekermann. "MetaConfigurator: A User-Friendly Tool for Editing Structured Data Files". In: *Datenbank-Spektrum* 24 (2024), pp. 161–169. DOI: 10.1007/s13222-024-00472-7 (cit. on pp. 24, 35).

[NEB+26] F. Neubauer, K. Endo, F. Bender, E. Ciftci, N. Hansen, S. Krause, B. Uekermann, J. Pleiss, et al. "Data-Model-Driven Management and Analysis of MOF Synthesis Data". In: (2026) (cit. on p. 53).

[NPU25] F. Neubauer, J. Pleiss, B. Uekermann. "Data Model Creation with MetaConfigurator". In: *Datenbanksysteme für Business, Technologie und Web (BTW 2025)*. Vol. P-361. Lecture Notes in Informatics (LNI), Proceedings. Gesellschaft für Informatik, Bonn, 2025. DOI: 10.18420/BTW2025-60. URL: https://dl.gi.de/handle/20.500.12116/45927 (cit. on pp. 24, 26, 27).

[NUP25] F. Neubauer, B. Uekermann, J. Pleiss. "AI-assisted JSON Schema Creation and Mapping". In: *28th ACM/IEEE Int. Conf. Model Driven Eng. Languages and Systems Companion (MODELS-C)*. 2025, pp. 79–83. DOI: 10.1109/MODELS-C68889.2025.00019 (cit. on p. 28).

[Ont] Ontotext. *GraphDB: RDF Database for Knowledge Graphs*. URL: https://www.ontotext.com/products/graphdb/ (cit. on p. 31).

[Proa] O. Project. *OpenRefine*. URL: https://openrefine.org/ (cit. on p. 31).

[Prob] R. Project. *RDFLib*. URL: https://rdflib.dev/ (cit. on p. 31).

[Proc] R. R. Project. *Redland RDF Libraries*. URL: http://librdf.org/ (cit. on p. 31).

[PS13] E. Prud'hommeaux, A. Seaborne. *SPARQL 1.1 Overview*. 2013. URL: https://www.w3.org/TR/sparql11-overview/ (cit. on pp. 21, 31).

[SBF98] R. Studer, V. R. Benjamins, D. Fensel. "Knowledge Engineering: Principles and Methods". In: *Data & Knowledge Engineering* 25.1–2 (1998), pp. 161–197. DOI: 10.1016/S0169-023X(97)00056-6 (cit. on p. 18).

[Sem] W. S. U. Semantic Computing Research Group. *SemTK: Semantic Toolkit*. URL: https://semtk.org/ (cit. on p. 32).

[SKL14] M. Sporny, G. Kellogg, M. Lanthaler. *JSON-LD 1.0: A JSON-based Serialization for Linked Data*. 2014. URL: https://www.w3.org/TR/json-ld/ (cit. on pp. 21, 33, 41).

[URR+26] J. F. Unger, A. Robens-Radermacher, S. M. Rosenbusch, D. Tyagi, D. Iglezakis, M. Jafarkhani. "A Platform for Validation and Verification of Models for Concrete and Concrete Structures". In: *Computational Modelling of Concrete and Concrete Structures*. Taylor & Francis, 2026. Chap. A platform for validation and verification of models for concrete and concrete structures. DOI: 10.1201/9781003660026-32. URL: https://www.researchgate.net/publication/401659007 (cit. on p. 15).

[Vc26] R. Verborgh, contributors. *SPARQL.js*. 2026. URL: https://www.npmjs.com/package/sparqljs (cit. on p. 46).

[W3C17] W3C. *Shapes Constraint Language (SHACL)*. 2017. URL: https://www.w3.org/TR/shacl/ (cit. on pp. 19, 31).

[WAHD20] A. Wright, H. Andrews, B. Hutton, G. Dennis. *JSON Schema: A Media Type for Describing JSON Documents*. IETF Draft. 2020. URL: https://json-schema.org/draft/2020-12/ (cit. on p. 16).

All links were last followed on March 10, 2026.

A Appendix

Erklärung

Ich versichere, diese Arbeit selbstständig verfasst zu haben. Ich habe keine anderen als die angegebenen Quellen benutzt und alle wörtlich oder sinngemäß aus anderen Werken übernommene Aussagen als solche gekennzeichnet. Weder diese Arbeit noch wesentliche Teile daraus waren bisher Gegenstand eines anderen Prüfungsverfahrens. Ich habe diese Arbeit bisher weder teilweise noch vollständig veröffentlicht. Das elektronische Exemplar stimmt mit allen eingereichten Exemplaren überein.

Ort, Datum, Unterschrift